Lavorare con un coding agent: contesto, piano, diff, verifica

Modulo 4 · Lezione 2

Dalla trace al vostro terminale

LA LEZIONE SCORSA · LETTA DA FUORI

Una sessione di Claude Code sul progetto Valtesa: **2 richieste, 12 chiamate**.

La correzione di `massimale()`, che ignorava la data della circolare: la nota 1043 passa da «fuori massimale» a «ok»; le note fuori massimale scendono da **312** a **293**.

OGGI · GUIDATA DA VOI

Lo stesso progetto, nella cartella della lezione. Il coding agent lo guidate voi.

Nella cartella c'è anche l'assistente di Valtesa (la versione del docente, da Modulo 3 · Lezione 12), con i documenti e le tabelle dell'ufficio. Oggi si lavora su `rimborsi.py`; l'assistente si esplora in Modulo 4 · Lezione 3.

> Da oggi il codice lo scrive il coding agent. Voi **dite cosa volete**, **leggete il piano**, **leggete il diff**, **verificate**.

Le note spese e i massimali

UNA NOTA SPESE

Una spesa di un dipendente in trasferta, da rimborsare. Una riga per nota in `dati/note_spese.csv`, 1.500 note:

```
id,dipendente,voce,citta,giorno,importo,…
1000,Luca Ferri,hotel,Milano,2026-04-18,124.26,…
1001,Chiara Lombardi,cena,Torino,2026-06-05,25.62,…
```

UN MASSIMALE

Quanto si rimborsa al massimo, per voce e città, da una data. In `dati/massimali.csv`:

```
voce,citta,massimale,valido_dal
cena,Milano,45,2026-01-01
cena,Milano,55,2026-07-01
hotel,Torino,110,2026-01-01
```

Una nota è **fuori massimale** se l'importo supera il massimale in vigore quel giorno, per quella voce e quella città: per esempio un hotel a Torino da 134,02 € contro 110 €. Oggi sono **293 su 1.500**.

Dal gestionale arrivano anche altri export. `dati/export-ottobre.csv`: 190 note, **16 di Bologna**, una città che in `massimali.csv` non c'è.



Agenda

	Blocco	Tempo
	Dalla trace al vostro terminale	7′
	Verifica del setup	5′
	Vibe coding e revisione	5′
	`AGENTS.md`, modalità piano, permessi	9′
	Una funzionalità ampia, dal piano ai test	22′
	Git come rete e sei controlli su un diff	6′
	`rimborsi.py`, il file dell'esercizio	3′
	Una modifica verificata · `CONSEGNA.md`	33′
	Rientro: una riga del vostro diff	15′
	Sessioni lunghe	10′
	La prossima lezione	5′

Cartella di lavoro: `modulo4-agenti/l02_coding-agent/scripts/`

Verifica del setup

FATE VOI — ognuno sul proprio computer

`modulo4-agenti/l02_coding-agent/scripts/` · 5 minuti

prima un `git pull`
poi si torna alle slide

Dalla vostra copia della repo del corso: `git pull`, poi `cd modulo4-agenti/l02_coding-agent/scripts`. Il coding agent aperto lì: nell'app di Claude una sessione **Local**, senza worktree (con il worktree lavora in un'altra cartella); da terminale `claude`. Un terminale nella stessa cartella, per `pytest` e git, con il venv attivo: `source ../../.venv/bin/activate` · PowerShell `..\..\.venv\Scripts\Activate.ps1`; il prompt allora comincia con `(.venv)`.

1. Il coding agent parte: nell'app risponde a un messaggio; da terminale `/status` mostra il vostro account · con Codex `codex login status`
2. I test: `pytest` → `11 passed`
3. Il file di contesto viene letto: chiedete all'agente «Senza aprire file: quale modello va usato in questo progetto?» → `gpt-5.6-luna`, il modello che usa il codice del progetto (non quello dell'agente). Se non lo sa, c'è un `CLAUDE.md` nella cartella o sopra: spostatelo o cancellatelo

Chi non parte: la sezione «Se il setup non parte» del `README.md` della cartella, e intanto segue sul proiettore.

`FATTO QUANDO` · `pytest` dice `11 passed` e l'agente risponde `gpt-5.6-luna`

Due modi di usare un coding agent

«There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. […] I "Accept All" always, I don't read the diffs anymore.»

Vibe coding: accetto tutto quello che scrive il modello, i diff non li leggo più.

Andrej Karpathy · 2 febbraio 2025

«If an LLM wrote every line of your code but you've reviewed, tested and understood it all, that's not vibe coding in my book - that's using an LLM as a typing assistant.»

Se il codice l'ha scritto il modello ma l'avete rivisto, testato e capito, non è vibe coding.

Simon Willison · 6 marzo 2025

> Hanno senso tutte e due: nel modulo vedremo casi in cui si fa la prima e casi in cui si fa la seconda. Oggi la **seconda**.



Il ciclo in quattro passi

1 · CONTESTO

`AGENTS.md` dice come si lancia, come si testa, cosa non toccare

voi: tenete una regola solo se sapete dire perché c'è

2 · PIANO

La richiesta in modalità piano: l'agente legge e propone, non modifica

voi: leggete il piano e lo correggete prima di dire sì

3 · DIFF

`git status` e `git diff` contro l'ultimo commit, sei controlli

voi: leggete il diff, non solo il messaggio finale

4 · VERIFICA

I test li lanciate voi; un numero lo controllate a mano

voi: lanciate i comandi e guardate l'output

Ogni passo può rimandare indietro: un **piano sbagliato** si corregge prima di dire sì; un **diff sbagliato** si butta con `git restore` (i file nuovi si cancellano) e si rifà la richiesta.



Su cosa e come: la cartella e le istruzioni

SU COSA · LA CARTELLA

```
rimborsi.py
assistente.py
dati/
ufficio/
tests/
```

Il codice, i dati, i documenti, i test. L'agente li **legge da sé**, quando gli servono.

COME · LE ISTRUZIONI

`AGENTS.md` — `in ogni sessione`
le regole del progetto: quale Python, come si testa, cosa non toccare

la richiesta — `questa volta`
cosa fare adesso e come verificarlo

le skill — `quando servono`
procedure da richiamare: Modulo 4 · Lezione 8

> Leggendo la cartella l'agente sa **cosa c'è**. Che `dati/` non si tocca a mano, o che i test esistenti non si cambiano per farli passare, nel codice e nei dati non c'è: **va scritto**, in un file che arriva in ogni sessione. Per questo serve `AGENTS.md`.

Cosa va in `AGENTS.md`

`AGENTS.md` è un file di testo nella cartella del progetto, con le istruzioni per il coding agent: lo carica all'avvio di ogni sessione.

```
## Regole del corso (non cambiarle): le righe fisse, già nella
cartella · venv · versioni · modello · async · niente CLAUDE.md

## Il progetto · un esempio: la soluzione del docente, accorciata
- rimborsi.py controlla le note spese di dati/note_spese.csv contro
dati/massimali.csv […] l'ultima riga oggi è «1500 note spese, 293
fuori massimale».
- assistente.py è l'assistente per i dipendenti […] chiama le API.

Come si testa: pytest dalla cartella del progetto, prima e dopo ogni
modifica.

Cosa non toccare: i file in dati/ sono export del gestionale […]; i
test esistenti non si cambiano per farli passare […].
```

Non ci va: il codice spiegato per esteso (l'agente lo legge da sé) e le chiavi. Ci va quello che il codice non dice: l'ultima riga di oggi, cosa non toccare.

Token all'avvio, con `/context`:



Il file completo vale **1k token** ed è nel contesto di ogni chiamata.

Quale file di contesto legge l'agente

Claude Code

«By default, Claude reads `AGENTS.md` only when you have no `CLAUDE.md` in your working directory or above it.»

Legge AGENTS.md solo se nella cartella o sopra non c'è un CLAUDE.md.

`/init` scrive un `CLAUDE.md`. Sul progetto, `/context`:

```
Memory files
AGENTS.md · 1k token
```

```
con un CLAUDE.md di 7 righe
CLAUDE.md · 57 token
```

L'agente risponde come sempre, ma le vostre istruzioni non arrivano.

Codex

«Codex concatenates files from the root down, joining them with blank lines. Files closer to your current directory override earlier guidance because they appear later in the combined prompt.»

Mette insieme gli AGENTS.md dalla radice alla cartella corrente; il più vicino vale di più, perché arriva per ultimo.

> Usiamo `AGENTS.md` perché lo leggono tutti e due. Per questo le righe fisse dicono: niente `CLAUDE.md`, neanche con `/init`.



La modalità di partenza e la modalità piano

Claude Code da terminale, dalla 2.1.283, parte in `auto`: «A separate classifier model reviews actions before they run.»

Un modello a parte controlla le azioni prima che partano.

Non chiede quasi mai conferma. Da terminale si cambia con **Shift+Tab**:

- `auto mode on` · un altro modello approva
- `manual mode on` · senza chiedere passano solo le letture
- `accept edits on` · passano anche le modifiche ai file
- `plan mode on` · legge e propone

Nell'app (la scheda Code di Claude) Shift+Tab non vale: la modalità si sceglie dal **selettore accanto al tasto di invio** (Cmd+Shift+M, su Windows Ctrl+Shift+M): Manual · Accept edits · Plan · Auto.

«Claude reads files, runs shell commands to explore, and writes a plan, but does not edit your source.»

Legge, esplora, scrive un piano; non modifica.

Da terminale ci si entra con Shift+Tab, o scrivendo `/plan` davanti alla richiesta. A piano pronto:

- `Yes, and use auto mode` · sì, poi approva l'altro modello
- `Yes, manually approve edits` · sì, e approvate voi ogni modifica
- `No, keep planning` · no: scrivete la correzione

Nell'app: Plan nel selettore; a piano pronto, le stesse tre risposte.

Codex, in una cartella con git, parte in `Auto`: legge, modifica e lancia comandi senza chiedere. I suoi comandi sono nella Slide 25.

`OGGI` · la richiesta in modalità piano, poi le modifiche approvate a mano («Yes, manually approve edits»): vedete passare ogni modifica

Demo: `valida.py` dal contesto ai test

GUARDATE IL DOCENTE — lavora in Claude Code, voi non lo rifate

`modulo4-agenti/l02_coding-agent/scripts/` · branch `demo-valida` · 22 minuti

dopo la verifica
si torna alle slide

1	Un branch per la demo e lo stato pulito: `git switch -c demo-valida`, `git status --short .` vuoto	
2	Il contesto: l'agente scrive la parte sul progetto di `AGENTS.md`; la si legge, un commit, una sessione nuova	Slide 13
3	La richiesta, in modalità Plan: nell'app il selettore accanto al tasto di invio, da terminale Shift+Tab	Slide 14
4	Il piano: si legge, poi si corregge una cosa prima di dire sì	Slide 15-16
5	Le modifiche approvate a mano, una per una; poi il diff	Slide 17
6	La verifica: i criteri uno per uno, dal terminale	Slide 18

Le Slide 13-18 accompagnano la demo e dicono cosa guardare mentre l'agente lavora. Se la demo non parte: la sessione registrata della stessa richiesta (con le modifiche accettate senza conferma), in `modulo4-agenti/l02_coding-agent/sessione-registrata/trace-valida.md`.

Il contesto: la parte sul progetto di `AGENTS.md`

DEMO DEL DOCENTE

Nella cartella `AGENTS.md` ha solo le regole del corso; la parte sul progetto la scrive l'agente:

```
Aggiungi in AGENTS.md, sotto le regole del corso, una sezione "## Il progetto": come si lancia, come si testa, cosa non toccare. Non cambiare le regole del corso.
```

I FATTI

Un comando che ha scritto lo si prova: `pytest`, `python rimborsi.py`. Nelle nostre prove i fatti erano giusti, 3 volte su 3.

LE REGOLE

Nelle nostre prove l'agente ha aggiunto ogni volta regole non richieste, come «`README.md` e `CONSEGNA.md` non si toccano». Una regola si tiene **solo se sapete dire perché c'è**. Questa ha senso se non volete il README cambiato senza averlo chiesto.

Poi `git add .`, `git commit -m "AGENTS.md"` e una sessione nuova (da terminale `/clear`), che carica il file nuovo: la richiesta di `valida.py` parte da qui.

IN CLAUDE CODE · la richiesta, con le modifiche approvate a mano; poi il diff di AGENTS.md



La richiesta: `valida.py`

DEMO DEL DOCENTE · la guardate, non la fate

La richiesta che il docente incolla in modalità Plan, accorciata:

```
Serve uno strumento nuovo, valida.py, che controlla un export del gestionale prima di darlo a rimborsi.py. Si lancia così: python valida.py dati/export-ottobre.csv

Criteri di accettazione:
1. controlla che il file abbia tutte le colonne di dati/note_spese.csv;
2. segnala le righe con: data non valida, importo non numerico o non positivo, id duplicato, voce sconosciuta, città senza massimali;
3. scrive un rapporto in output/, con una riga per errore e in cima un riepilogo per tipo;
4. esce con codice 1 se ci sono errori, 0 se non ce ne sono;
5. su dati/note_spese.csv non trova errori; su dati/export-ottobre.csv trova le 16 note di Bologna;
6. i test in tests/test_valida.py coprono ogni tipo di errore con file costruiti apposta, senza toccare dati/.

Prima fammi vedere il piano.
```

> I criteri di accettazione sono la **verifica, scritta prima**: dopo si controllano uno per uno.

IN CLAUDE CODE · Plan nel selettore, poi la richiesta intera



Il piano: cosa si guarda

DEMO DEL DOCENTE

In modalità Plan l'agente **legge e propone**: non tocca i file del progetto.

Mentre scrive il piano si guardano i comandi che lancia: **solo letture**.

Nel piano si cerca:

- quali file **crea** e quali **tocca**, e cosa dice che non tocca (`rimborsi.py`, `dati/`, i test esistenti)
- da dove prende voci e città: da `massimali.csv` o da una lista scritta nel codice
- un test per ogni tipo di errore, su file costruiti apposta
- come pensa di verificare, criterio per criterio
- le **domande** che fa: si leggono prima di dire sì

IN CLAUDE CODE · i comandi di lettura, poi il piano dall'alto



La correzione del piano

DEMO DEL DOCENTE

Prima di dire sì il docente cambia **una cosa del piano**, a sua scelta, purché poi si possa controllare nel diff.

LA CORREZIONE

`No, keep planning`, poi la correzione scritta.
L'agente rifà il piano, e lo si rilegge.

L'APPROVAZIONE

`Yes, manually approve edits`: il piano nuovo è approvato, e ogni modifica chiede il permesso.
Nell'app e da terminale le risposte sono le stesse.

QUANTO COSTA

Correggere il piano costa **una riga**. Correggere il diff costa rifare **la richiesta e la verifica**.

IN CLAUDE CODE · la correzione, il piano nuovo, l'approvazione

Il diff

DEMO DEL DOCENTE

Ogni modifica passa dal docente: l'agente la mostra e aspetta. Anche i comandi che vuole lanciare chiedono il permesso.

ATTESO, DALLA RICHIESTA

```
$ git status --short .
?? tests/test_valida.py
?? valida.py
```

Nell'app: l'indicatore delle righe (`+N -M`) apre il diff, file per file. Da terminale: `git status --short .` e i file nuovi aperti: un file nuovo non compare in `git diff`.

Cosa si guarda:

- solo file nuovi: `valida.py` e `tests/test_valida.py`
- `rimborsi.py`, `dati/` e i test esistenti non toccati
- dove sta la correzione del piano, nel codice o nel rapporto

IN CLAUDE CODE · le modifiche approvate una per una, poi il diff



La verifica: criteri, test, punti a rischio

DEMO DEL DOCENTE

Dal terminale, non dall'agente. Un diff di centinaia di righe non si legge riga per riga; quello dell'esercizio, di poche righe, sì.

Criterio	Come	Atteso
1-2, 6	i nomi dei test (`pytest -v tests/test_valida.py`), poi `pytest -q`	un test per tipo di errore · tutti verdi, gli 11 di prima compresi
3	aprire il rapporto in `output/`	riepilogo in cima, una riga per errore
4-5	`python valida.py dati/note_spese.csv`, poi `echo $?` (il codice di uscita; PowerShell `$LASTEXITCODE`)	nessun errore · 0
4-5	lo stesso su `dati/export-ottobre.csv`	le 16 note di Bologna · 1
6	`git status --short .`	solo i file nuovi
	`python rimborsi.py`	l'ultima riga resta «1500 note spese, 293 fuori massimale»

NEL TERMINALE · i comandi della tabella, con il venv attivo

LA CORREZIONE

quella chiesta nel piano: si cerca nel codice o nel rapporto

PUNTI A RISCHIO, LETTI A MANO

le scelte che l'agente ha fatto senza che nessuno le chiedesse, nel piano o nel resoconto finale. Il resto è coperto dai criteri e dai test

NON VERIFICATO

un export con errori di altri tipi, come date o duplicati (in `export-ottobre.csv` c'è solo Bologna) · il funzionamento su Windows



Git come rete di sicurezza

1 · UN BRANCH

```
git switch -c l02-lavoro
```

dopo il `git pull`

→

2 · TENERE

```
git add .
git commit -m "…"
```

da `scripts/`

→

3 · DOPO UNA RICHIESTA

```
git status --short .
git diff --stat .
git diff .
```

quali file (anche i nuovi, `??`) · quante righe · le righe; si esce con q

→

4 · BUTTARE

```
git restore <file>
```

i file nuovi si cancellano a mano

Perché il branch: con commit vostri su `main`, il `git pull` della lezione dopo si ferma: «Need to specify how to reconcile divergent branches» *(git non sa come unire i vostri commit con quelli nuovi)*. Alla lezione dopo: `git switch main`, poi `git pull`.

Perché il punto: `.` vuol dire «solo questa cartella», in `git status` come in `git diff`. Nelle cartelle delle altre lezioni avete file cambiati negli esercizi di Modulo 2 e 3, che qui non c'entrano.

> A lavoro finito, tutto quello che avete fatto sul branch: `git diff main --stat .`



Sei controlli su un diff

1 · File toccati
sono quelli che vi aspettate?
`git status --short .`

2 · Righe tolte o cambiate
che non avevate chiesto: le righe con `-`

3 · Test cambiati
perché? `git diff tests/`

4 · Dipendenze nuove
`requirements.txt`, import nuovi

5 · Valori scritti a mano
numeri, nomi, percorsi che dovrebbero stare nei dati

6 · Cosa non è stato lanciato
i test? il programma sui dati veri?

In 13 prove con una richiesta più grande (far leggere l'export di ottobre a `rimborsi.py`, senza dire di lasciare l'ultima riga com'era): README modificato senza richiesta **3** volte, ultima riga cambiata **4** volte. Un diff vero:

```
- print(f"\n{len(note)} note spese, {fuori} fuori massimale")
+ print(f"\n{len(note)} note spese, {fuori} fuori massimale, {da_verificare} da verificare a mano")
```

Se l'agente cambia quell'ultima riga, i controlli sull'output di `python rimborsi.py`, a mano o in uno script, non tornano più.

Test cambiati per farli passare

1. La richiesta: «Fai restituire a `controlla()` anche il massimale usato. Poi fai passare i test.»
2. Prima `controlla()` restituiva `True` o `False`. Ora una coppia, per esempio `(False, 40.0)`.
3. Per Python una coppia è sempre «vera»: `assert not (False, 40.0)` fallisce, `assert (False, 40.0)` passa. Quattro test falliscono, **due passano sempre**.

L'AGENTE, NELLE NOSTRE PROVE

```
- assert not controlla({…taxi, Roma, 52.00})
+ assert controlla({…taxi, Roma, 52.00})
+      == (False, 40.0)
```

In 3 prove su 3 riscrive tutti i test, più stretti: ora controllano esito e massimale. Cambiati per un motivo giusto.

COSTRUITO DA NOI · L'AGENTE NON L'HA FATTO

```
  assert controlla({…pranzo, Torino})
- assert not controlla({…taxi, Roma})
+ assert not controlla({…taxi, Roma})[0]
```

Corregge solo i quattro test che fallivano: `pytest` verde. Ma la riga evidenziata non è cambiata, e ora **passa sempre**. Se rompiamo apposta il codice, 2 test su 6 restano verdi.

> Un test cambiato si rilegge per intero, anche nelle righe che il diff non tocca: **controlla ancora quello che doveva?**



`rimborsi.py`, il file dell'esercizio

```
NOTE_SPESE = CARTELLA / "dati" / "note_spese.csv"

def massimale(voce, citta, giorno):
    # la riga di massimali.csv in vigore nel giorno
    # se non c'è: raise KeyError

def controlla(nota):
    # True se l'importo sta nel massimale

def main():
    for nota in note:
        limite = massimale( ... )
        ok = float(nota["importo"]) ⩽ limite
        print( ... )  # una riga per nota
    print(f"\n{len(note)} note spese, {fuori} fuori massimale")
```

`massimale( )` cerca in `massimali.csv` la riga in vigore per voce, città e giorno della nota.

I 6 test di `tests/test_rimborsi.py` controllano `controlla( )`; `main( )` rifà il conto da sé e non è coperto dai test.

`main( )` stampa **una riga per nota**, che le modifiche 1 e 2 del menu (Slide 24) filtrano o riassumono, e in fondo l'ultima riga, che non deve cambiare.

```
$ python rimborsi.py
 1000  Luca Ferri         hotel    Milano   2026-04-18   124.26  ok
 1011  Stefano Greco      hotel    Torino   2026-03-11   134.02  FUORI MASSIMALE (110.00)
…
1500 note spese, 293 fuori massimale
```




Esercizio: una modifica verificata

FATE VOI — ognuno sul proprio progetto, con il coding agent

`modulo4-agenti/l02_coding-agent/scripts/rimborsi.py` · `CONSEGNA.md` · 33 minuti

poi si torna alle slide

1 · Il branch: `git switch -c l02-lavoro`; `git status --short .` pulito

2 · Una modifica di `rimborsi.py` dal menu, richiesta in modalità piano; nella richiesta scrivete come la verificherete e cosa non va toccato (oggi `AGENTS.md` ha solo le regole del corso)

3 · Il piano: se va corretto, «No, keep planning» e la correzione; se va bene, «Yes, manually approve edits». Nell'app le risposte sono le stesse

4 · Il diff, con i sei controlli; poi un commit, che salva il lavoro anche se non arrivate al 5

5 · `pytest` e il numero li controllate voi, dal terminale (se l'agente chiede di lanciare i test, ditegli sì: poi li rilanciate voi). Se non ci arrivate, va bene.

`FATTO QUANDO` · dopo il commit, `git diff main --stat .` mostra solo i file che vi aspettate (`rimborsi.py`, i test se servono) e del diff sapete dire cosa fa ogni parte · se ci arrivate: `pytest` verde e il numero che torna con il vostro controllo

Il menu delle modifiche

Una a scelta. Il risultato lo verificate voi, con il controllo indicato (con PowerShell, i comandi sono in `CONSEGNA.md`).

1

```
python rimborsi.py --dipendente "Luca Ferri"
```

Solo le note di quella persona, e quante sono fuori massimale.

le note: `grep -c "Luca Ferri" dati/note_spese.csv`
i fuori massimale: `python rimborsi.py | grep "Luca Ferri" | grep -c FUORI`

2

```
python rimborsi.py
```

Un riepilogo per città, note e fuori massimale, **prima** della riga dei totali, che resta l'ultima.

una città: `grep -c ",Milano," dati/note_spese.csv`
poi: le somme per città tornano ai totali

Per tutte e due: senza argomenti l'ultima riga resta `1500 note spese, 293 fuori massimale`. **Se finite prima**: la parte sul progetto di `AGENTS.md`, come nella demo; poi l'altra modifica. Tutte le estensioni sono in `CONSEGNA.md`.



I comandi: Claude Code e Codex

	Claude Code	Codex
Avviare, nella cartella	`claude`	`codex`
Lo stato	`/status`	`/status`
Modalità piano	Shift+Tab o `/plan` davanti alla richiesta · app: Plan nel selettore	Shift+Tab o `/plan`
Cambiare i permessi	Shift+Tab · app: il selettore (Cmd/Ctrl+Shift+M)	`/permissions`
Vedere le modifiche	`/diff` · app: l'indicatore `+N -M`	`/diff`
Il contesto	`/context`	`/status`
Compattare	`/compact`	`/compact`
Ricominciare con il contesto vuoto	`/clear` · app: una sessione nuova	`/new`
Riprendere l'ultima sessione	`claude --continue`	`codex resume --last`
Scegliere una sessione	`/resume`	`/resume`
Quanto avete usato dell'abbonamento	`/usage` · app: l'anello accanto al menu del modello	`/status`

Tutti e due leggono `AGENTS.md`; Claude Code solo se non c'è un `CLAUDE.md`.



Rientro: una riga del vostro diff

Chi vuole mostra il suo `git diff main --stat .` e spiega una riga di `git diff main .`: cosa fa, perché c'è.

1. Quale file ha toccato l'agente che **non vi aspettavate**?
2. Cosa avete **corretto nel piano**, o perché andava bene così?
3. Cosa **non** avete verificato?



Cosa c’è nel contesto di una sessione

Con Opus 5.5 (finestra di contesto da 1M token) e l'`AGENTS.md` completo, `/context` all'avvio dà **21,6k token**; con un altro modello i numeri cambiano.



AGENTS.md

altri **19,4k** di tool di cui il modello vede solo il nome, caricati se servono

33k tenuti liberi per la compattazione automatica

poi cresce a ogni **file letto** e a ogni **output di comando**

> Con la richiesta di `valida.py`, in una sessione registrata: in 10 chiamate l'agente ha riletto dalla cache circa **345.000 token** (la cache: Modulo 4 · Lezione 1).

Compattare, ripulire, riprendere

COMPATTARE

`/compact`

Sostituisce la conversazione con un riassunto e libera spazio. Si può dire cosa tenere: `/compact tieni le decisioni sul formato del rapporto`.

Scatta anche da sola: con Opus 5.5 e la finestra da 1M, a circa 967K token. Costa: per riassumere rilegge tutta la conversazione.

RIPULIRE

`/clear` · Codex `/new`

Una conversazione nuova, con il contesto vuoto. Non costa niente.

Il file di contesto si ricarica dal disco.

RIPRENDERE

`claude --continue` · `/resume`

`--continue` riapre l'ultima conversazione della cartella: la storia intera, il modello, la modalità dei permessi (non quella piano). `/resume` sceglie una sessione.

Codex: `codex resume --last`, `/resume`

Quando ripartire da zero

Compito nuovo, che non c'entra con il precedente	«`/clear`: reset context between unrelated tasks. Long sessions with irrelevant context can reduce performance.» *Tra compiti non collegati si azzera il contesto: una sessione lunga con contesto che non serve può peggiorare le risposte.*
Stesso compito, conversazione lunga	`/compact` a una pausa naturale, dicendo cosa tenere, invece di aspettare che scatti da sola a metà lavoro.
Avete corretto l'agente più di due volte sulla stessa cosa	«If you've corrected Claude more than twice on the same issue in one session, the context is cluttered with failed approaches. Run `/clear` and start fresh with a more specific prompt that incorporates what you learned.» *Il contesto è pieno di tentativi sbagliati: si riparte con una richiesta più specifica, che include quello che avete capito.*

Codex: «Keep one chat per coherent unit of work.»

Una conversazione per ogni unità di lavoro coerente.



La prossima lezione

Modulo 4 · Lezione 3, lab di 4 ore: codice non vostro, con il coding agent. Quattro lavori sul progetto Valtesa.

1
Orientarsi nella parte che oggi non avete toccato: l'assistente

2
Un bug segnalato dall'ufficio, da riprodurre e correggere

3
Una modifica di un collega, da rivedere

4
Uno strumento intero, da un ticket

Per partire: `git switch main`, `git pull`, poi il `.env` (copiato da quello di oggi) e `python indicizza.py` nella cartella nuova, che ha già un `AGENTS.md` completo. Il vostro lavoro di oggi resta sul branch `l02-lavoro`.

Grazie

prossima lezione: Modulo 4 · Lezione 3 — Lab: codice non vostro, con il coding agent

